

Course Enrollment — Spring Boot + Angular (Integrated) Guide

Overview

This guide shows how to create a simple integrated Spring Boot + Angular project named "course-enrollment" that demonstrates:

- Showing available courses
- Enrolling in a course
- Viewing enrolled courses

Project structure (final)

```
course-enrollment/  
  ■■■■ src/  
  ■   ■■■■ main/  
  ■     ■■■■ java/com/example/course/  
  ■       ■   ■■■■ CourseController.java  
  ■       ■   ■■■■ Course.java  
  ■       ■   ■■■■ CourseService.java  
  ■       ■■■■ resources/  
  ■         ■■■■ public/ <-- Angular build files  
  ■■■■ ui/ <-- Angular project  
  ■■■■ build.gradle or pom.xml  
  ■■■■ README.md
```

1) Create Spring Boot project

- Use Spring Initializr or CLI:
 spring init --dependencies=web course-enrollment
- Or create via start.spring.io (choose 'Web' dependency)

2) Implement backend model and controller

File: src/main/java/com/example/course/Course.java

```
-----  
public class Course {  
    private Long id;  
    private String name;  
    private String description;  
    public Course() {}  
    public Course(Long id, String name, String description) {  
        this.id = id; this.name = name; this.description = description;  
    }  
    // getters & setters  
}
```

File: src/main/java/com/example/course/CourseController.java

```
-----  
@RestController  
@RequestMapping("/api/courses")  
public class CourseController {  
    private List<Course> courses = new ArrayList<>(Arrays.asList(  
        new Course(1L, "Java Basics", "Intro to Java"),  
        new Course(2L, "Spring Boot", "Building APIs"),  
        new Course(3L, "Angular Fundamentals", "Front-end dev")  
    ));  
    private List<Course> enrolled = new ArrayList<>();  
    @GetMapping public List<Course> getAllCourses(){ return courses; }  
    @PostMapping("/enroll/{id}") public String enroll(@PathVariable Long id) {  
        courses.stream().filter(c->c.getId().equals(id)).findFirst().ifPresent(enrolled::add);  
        return "Enrolled Successfully!";  
    }  
    @GetMapping("/enrolled") public List<Course> getEnrolledCourses(){ return enrolled; }  
}
```

3) Create Angular app inside the project

From project root:

- Use Angular CLI 18 (or npx)
npx @angular/cli@18 new ui --skip-git --routing=true --style=css

4) Configure Angular output

- Open ui/angular.json and set:

```
"outputPath": "../src/main/resources/public"
```

This ensures built files go into Spring Boot static resources.

5) Create components (inside ui)

- Generate components:
ng generate component course-list
ng generate component enrolled-course

6) Implement CourseList component

File: ui/src/app/course-list/course-list.component.ts

```
import { HttpClient } from '@angular/common/http';  
...  
ngOnInit(){ this.http.get<any[]>('/api/courses').subscribe(data=>this.courses=data); }  
enroll(id:number){ this.http.post('/api/courses/enroll/'+id,{}).subscribe(()=>alert('Enrolled!')); }  
-----
```

Template (course-list.component.html)

```
<ul>  
  <li *ngFor="let c of courses">  
    {{c.name}} - {{c.description}}  
    <button (click)="enroll(c.id)">Enroll</button>  
  </li>  
</ul>
```

7) Implement EnrolledCourse component

- Use GET /api/courses/enrolled and display enrolled list.

8) Wire HttpClientModule

- In ui/src/app/app.module.ts import HttpClientModule and add to imports.

9) index.html base href

- If deploying under a context path such as /course-enrollment/ set:
<base href="/course-enrollment/">
- If serving from root, set:
<base href="/">

10) Development workflow (local)

Option A (Spring Boot serves built files):

- In ui: ng build --watch
(writes to src/main/resources/public on change)
- In project root: ./mvnw spring-boot:run (or ./gradlew bootRun)
- Visit: http://localhost:8080/course-enrollment/ (or /)

Option B (frontend live reload):

- In ui: ng serve --proxy-config proxy.conf.json
- Proxy example (ui/proxy.conf.json):
{ "/api": { "target": "http://localhost:8080", "secure": false } }
- Visit: http://localhost:4200 (dev server)

11) Production build & Tomcat deployment

- From ui: export NODE_OPTIONS=--openssl-legacy-provider (if using Node 17+ with older webpack)
- Build production: ng build --configuration production --base-href /course-enrollment/
- Ensure build output is in src/main/resources/public
- Build WAR/JAR with Gradle or Maven:
./gradlew clean build (or mvn clean package)
- Rename/copy WAR to tomcat/webapps/COURSE-ENROLLMENT.war (context path must match base href)
- Visit: http://server:8080/course-enrollment/

12) Troubleshooting common issues

- Blank white page but favicon loads:
 - * Check browser DevTools → Network for missing main.*.js files (404)
 - * Ensure base href matches context path and filenames
- Build failure with Node crypto error:
 - * Use: `export NODE_OPTIONS=--openssl-legacy-provider`
 - * Or use Node 16 (`nvm install 16; nvm use 16`)
- Missing Angular files in WAR:
 - * Ensure Angular build runs and copies files to `src/main/resources/public` before packaging
 - * Automate with Gradle task to run frontend build before packaging

13) Example Gradle snippet to build frontend before packaging

```
-----  
task buildFrontend(type: Exec) {  
    workingDir 'ui'  
    commandLine 'npm', 'run', 'build'  
}  
processResources.dependsOn buildFrontend  
-----
```

14) Quick commands summary

- Dev build (watch): `cd ui; ng build --watch`
- Prod build: `cd ui; ng build --configuration production --base-href /course-enrollment/`
- Package: `./gradlew clean build`
- Deploy WAR: `cp build/libs/*.war /path/to/tomcat/webapps/`

End of guide.